Projekt kompetencyjny IOAD 2026/2027

Dokument wizji

PlayLink Multi-Region

Lobby w dwóch regionach chmury: routing, powtarzalne wdrożenie, eksperyment i próby awarii

Bartosz Kołaciński, nr indeksu: 251554
Mateusz Kosowski, nr indeksu: 251558
Nikodem Nowak, nr indeksu: 251598
Natan Ręklewski, nr indeksu: 252952
Jakub Rosiak, nr indeksu: 251620
Andrzej Krysiak, nr indeksu: 252945

Opiekun projektu: mgr inż. Bartłomiej Jencz

3 listopada 2026

Repozytorium: https://github.com/ioad-modul-sumatywny-26/playlink

Spis treści

1 Informacje o dokumencie **1**

1.1 Cel dokumentu 1

1.2 Definicje i skróty 1

2 Problem biznesowy **2**

2.1 Obecna sytuacja 2

2.2 Problem do rozwiązania 2

3 Wizja produktu **2**

3.1 Jedna idea, sześć odrębnych rezultatów 3

3.2 Zależności między filarami 3

4 Cele biznesowe **4**

5 Grupy użytkowników **4**

6 Zakres produktu **4**

6.1 Funkcjonalności w zakresie 4

6.2 Poza zakresem 6

7 Kluczowe wymagania biznesowe **7**

8 Główne procesy biznesowe **7**

9 Wymagania niefunkcjonalne **8**

10 Integracje **9**

11 Założenia i ograniczenia **10**

11.1 Założenia 10

11.2 Ograniczenia 11

11.3 Wybór chmury i koszty 11

11.3.1 Dlaczego nie ma trzeciego serwera 11

11.3.2 Co bierzemy z Cloudflare 12

11.3.3 Czego nie przenosimy na Cloudflare 12

11.3.4 Kontenery, sekrety, przepis 12

12 Ryzyka **14**

13 Kryteria sukcesu **14**

14 Roadmapa produktu **15**

14.1 Dziennik etapu 15

15 Korzyści biznesowe **16**

16 Załączniki **16**

1 Informacje o dokumencie

1.1 Cel dokumentu

Dokument przedstawia wizję rozwoju aplikacji PlayLink z pojedynczej instancji do platformy działającej w dwóch regionach chmury publicznej (docelowo dwa oddalone od siebie regiony, np. Europa i USA). Stos regionalny stawiamy na Microsoft Azure (kredyty studenckie); AWS jest alternatywą. Domena, tunele i ochrona przed atakami są na Cloudflare. Maszyny i kontenery, razem z bazą, są na Azure. Wybór regionu robi przeglądarka, bez trzeciego serwera (sekcja 11.3). Dokument opisuje problem biznesowy, wartość dla użytkowników, zakres, wymagania, ryzyka i kryteria sukcesu, uwzględniając zrealizowane rezultaty prac (PR 91, 92, 96, 97, 99): wdrożenie wieloregionowe EU/US za tunelami Cloudflare, routing z propozycją Stay/Switch w przeglądarce, zautomatyzowane wdrożenie pull-based z GHCR, eksperyment obciążeniowy F4, centralny stos obserwowalności (Grafana, VictoriaLogs, VictoriaMetrics) oraz dedykowany serwer Model Context Protocol (MCP).

1.2 Definicje i skróty

Termin	**Znaczenie**
LFG	*Looking For Group* — szukanie współgraczy.
Lobby / pokój	Krótkotrwała sesja gry z limitem miejsc, wygasająca po czasie.
Region	Osobne środowisko w chmurze (np. Frankfurt, N. Virginia) z kompletnym stosem aplikacji.
Stos regionalny	Frontend, backend i baza danych wdrożone w jednym regionie.
Routing użytkowników	Mechanizm wyboru regionu dla użytkownika (automatyczny lub ręczny).
Stay/Switch	Mechanizm propozycji zmiany regionu: jeśli zapisany region żyje, interfejs proponuje szybszy region banerem bez agresywnego przerywania sesji.
IaC	*Infrastructure as Code* — opis maszyn, bazy, DNS i sekretów w plikach w repozytorium. Jedno polecenie tworzy region; kolejny region to nowy plik parametrów, a nie klikanie w panelu.
Sonda regionu	Pomiar czasu odpowiedzi endpointu zdrowia, wykonywany w przeglądarce przed wyborem regionu.
RTT	*Round-Trip Time* — czas przejścia sygnału tam i z powrotem.
RTO / RPO	Maksymalny akceptowalny czas niedostępności / utraty danych po awarii.
p50, p95	Mediana i 95. percentyl czasu odpowiedzi.
WS	WebSocket — kanał czasu rzeczywistego (lista pokojów, czat).
MCP	*Model Context Protocol* — standard bezpiecznej komunikacji narzędziowej i udostępniania zasobów dla asystentów AI (Claude, Antigravity, Cursor).
VictoriaLogs / Metrics	Centralny system składowania i przeszukiwania logów (LogsQL) oraz metryk (PromQL) ze wszystkich regionów.

2 Problem biznesowy

2.1 Obecna sytuacja

Jak działa dziś. PlayLink to działająca aplikacja LFG (FastAPI, SvelteKit, PostgreSQL, Docker Compose) wdrażana automatycznie z gałęzi `main` na *jeden* serwer. Każdy pokój ma pole `lobby_location`, ale to tylko *etykieta* (10 predefiniowanych lokalizacji w `backend/config.py`); nie oznacza ona, że pokój obsługuje serwer w tym regionie. Wskaźnik ping w interfejsie mierzy czas do tego jednego backendu, więc nie mówi nic o jakości połączenia z lobby w innym regionie.

Jakie są problemy.

- Użytkownik z daleka od serwera ma wyższe opóźnienia przy każdej akcji (utworzenie pokoju, dołączenie, czat, odświeżenie listy).
- Lobby „w USA" i „w Europie" są rozróżnione tylko opisowo — nie da się zweryfikować, czy dwa regiony faktycznie pomagają graczom.
- Pojedynczy punkt awarii: niedostępność serwera wyłącza całą platformę.
- Stan czasu rzeczywistego jest trzymany w pamięci procesu (menedżery połączeń WebSocket i liczniki limitów czatu), więc obecna architektura nie skaluje się poziomo "za darmo".
- Wdrożenie jest opisane skryptem CI dla jednego hosta, a nie powtarzalnym szablonem infrastruktury.

Koszty i ryzyka. Wyższe opóźnienia zniechęcają graczy z odległych regionów (a w grach niszowych każdy odpadający gracz osłabia i tak małą społeczność). Awaria jedynego serwera oznacza całkowity brak usługi. Rozbudowa „na oko" o drugi region bez pomiarów mogłaby podwoić koszty bez udowodnionej korzyści.

2.2 Problem do rozwiązania

Nie wiemy, *czy i kiedy* wdrożenie w dwóch regionach realnie poprawia doświadczenie graczy, jaką cenę (koszt, złożoność, ograniczenia) trzeba za to zapłacić i jak zachowa się system, gdy jeden region przestanie działać. Projekt ma na te pytania odpowiedzieć **pomiarem**, a nie założeniem.

3 Wizja produktu

> Dla **graczy gier niszowych i retro z Europy i Ameryki Północnej**, którzy potrzebują **szybkiego, niezawodnego lobby blisko siebie**, produkt **PlayLink Multi-Region** jest **platformą LFG działającą w dwóch regionach chmury z automatycznym i ręcznym wyborem regionu**, która umożliwia **niskie opóźnienia, ciągłość działania przy awarii jednego regionu oraz oparte na pomiarach decyzje o tym, czy dodatkowy region jest opłacalny**.

3.1 Jedna idea, sześć odrębnych rezultatów

Wszystkie elementy poniżej służą jednej idei (dwa regiony), ale każdy odpowiada na *inne pytanie*, ma *inny produkt końcowy* i *inną miarę sukcesu*. Dzięki temu nie są sześcioma opisami tej samej rzeczy.

Filar	**Pytanie**	**Co zrealizowano**	**Miara**
F1 Regiony i lobby	Co dokładnie działa w regionie?	Dwa kompletne stosy (EU, US) na Azure z własnymi lobby za Cloudflare Tunnel	Oba regiony obsługują pełny przepływ: pokój, dołączenie, czat (PR 91)
F2 Routing	Skąd system wie, dokąd wysłać użytkownika?	Sonda opóźnień w przeglądarce, baner propozycji Stay/Switch i ręczny przełącznik	Użytkownik trafia do szybszego regionu; brak niechcianych przeskoczeń (PR 91, 92)
F3 Wdrożenie	Czy środowisko da się odtworzyć bez ręcznej pracy?	Szablony compose, skrypty `deploy/deploy.sh` i wdrożenie pull-based z GHCR na Azure VM	Czysty start regionu w kilka minut; automatyczna aktualizacja wydań (PR 91)
F4 Eksperyment	Czy dwa regiony są szybsze niż jeden?	Dedykowany runner i scenariusze w `experiments/f4/`; zebrane pomiary 1 vs 2 regionów	Zmierzona różnica p50/p95, błędów i czasu synchronizacji WebSocket (PR 92)
F5 Awarie	Co się dzieje, gdy region pada?	Scenariusze awarii, procedury runbookowe, automatyczne przełączenie przy braku odpowiedzi	Czas wykrycia, ciągłość działania drugiego regionu i brak wpływu na bazę
F6 Analiza	Kiedy to się opłaca?	Raport z wnioskami: korzyści, koszty maszyn w Azure, transfer i podział graczy	Rekomendacja poparta rzeczywistym rachunkiem i pomiarami F4

Tabela 1: Odrębność sześciu filarów projektu oraz ich stan realizacji.

3.2 Zależności między filarami

F1 jest fundamentem (bez regionów nie ma czego routować). F2 i F3 są niezależnymi strumieniami pracy nad F1 (kod aplikacji vs. infrastruktura). F4 wymaga F1–F3 i produkuje dane dla F6. F5 wymaga F1–F3, a jej wyniki są osobnym wejściem do F6. Dzięki temu zespół może pracować równolegle, a F4–F6 są etapem *badawczym*, nie kolejnym etapem budowy.

4 Cele biznesowe

ID	Cel	Sposób weryfikacji
CB-1	Skrócić opóźnienia dla graczy z obu kontynentów w porównaniu z jednym regionem	Eksperyment F4: p95 dla użytkowników z USA i z Europy
CB-2	Zachować możliwość gry mimo awarii jednego regionu	Próby awarii F5: czas odzyskania, dostępność drugiego regionu
CB-3	Umożliwić odtworzenie środowiska jednym poleceniem	F3: czysty start w nowym koncie lub regionie
CB-4	Poznać koszt i ograniczenia modelu dwuregionowego	F6: tabela kosztów i ograniczeń
CB-5	Utrzymać dotychczasowe wymagania jakościowe (WN-01 p95 $<$ 200 ms, WN-02 $<$ 1000 ms)	Regresja testów i pomiary w obu regionach

5 Grupy użytkowników

Grupa	Charakterystyka	Potrzeba w wizji
Gracz z Europy	Najliczniejsza grupa dziś; blisko obecnego serwera	Brak pogorszenia jakości; możliwość dołączenia do lobby w USA
Gracz z USA	Dziś obsługiwany z daleka	Lobby blisko siebie i niższe opóźnienia
Gracz „międzyregionalny”	Gra w tytuły o małej społeczności, chce większej puli graczy	Ręczny wybór regionu niezależnie od lokalizacji
Administrator	Moderuje pokoje i kategorie gier	Zarządzanie w każdym regionie
Operator / zespół	Wdraża i obserwuje system	Jeden szablon, metryki, alarmy

6 Zakres produktu

6.1 Funkcjonalności w zakresie

F1 — Dwa regiony chmury z działającymi lobby. Zrealizowano dwa niezależne stosy regionalne (Europa i USA, projekty `playlink-eu` i `playlink-us` w PR 91), każdy z frontendem, backendem i bazą PostgreSQL. **Lobby jest regionalne**: pokój istnieje w bazie regionu, w którym go utworzono. To świadoma decyzja architektoniczna — stan czasu rzeczywistego jest w pamięci procesu, a pokoje są krótkotrwałe, więc replikacja między regionami dawałaby dużą złożoność za niewielki zysk. Maszyny wirtualne nie posiadają otwartych portów publicznych — ruch w całości obsługuje Cloudflare Tunnel. Endpointy `/healthz` i `/readyz` weryfikują stan backendu i bazy, a rate limiter opiera się na nagłówku `CF-Connecting-IP`. *Wspólna* jest tożsamość (konto powstaje przy pierwszym podpisie, a klucz nie opuszcza przeglądarki), *regionalne* są pokoje, czat i listy.

F2 — Kierowanie użytkowników. Adres strony wskazuje na jeden z regionów za pośrednictwem Cloudflare. Skrypt w odpowiedzi mierzy czas do endpointu zdrowia każdego regionu i otwiera lobby w szybszym, który odpowiedział. Osobnego serwera routingu nie ma. Wybór regionu jest zapamiętywany w cookie (`playlink.region`). W ramach PR 92 wdrożono dopracowany mechanizm **Stay/Switch** (komponent `RegionProposalBanner`): przy kolejnych wejściach gracz wraca do zapisanego regionu, a gdy pomiar w tle wskaże, że drugi region jest szybszy, interfejs wyświetla nienatarczywy baner z propozycją zmiany. Sesja nie jest przerywana samoczynnie. Dopiero gdy zapisany region nie odpowiada, następuje automatyczny failover na żywy region. Ręczny przełącznik w nagłówku pozwala na swobodną zmianę w dowolnym momencie. Adresy API i WS są dynamicznie konfigurowane pod wybrany region.

F3 — Powtarzalne wdrożenie i automatyzacja wydań. Zamiast ręcznego wdrożenia czy bezpośredniego dostępu SSH z potoku CI, wdrożono zautomatyzowany, bezpieczny model pull-based (PR 91):

- Skrypt `deploy/deploy.sh <eu|us> [tag]` umożliwia natychmiastowe uruchomienie parametryzowanego stosu regionalnego (lokalnie lub w chmurze) z wygenerowanym środowiskiem `.run/<region>.env`.
- Potok wydań `.github/workflows/release.yml` buduje wieloarchitekturowe obrazy i publikuje je w GitHub Packages (GHCR) po oznaczeniu commita tagiem wydania.
- Maszyny Azure mają skonfigurowany timer systemd (`deploy/update.sh` co 2 minuty), który automatycznie pobiera najnowszy release (najpierw EU, po weryfikacji zdrowia US). W razie błędu healthchecku skrypt automatycznie wycofuje wdrożenie (rollback).
- Dla wygody prac deweloperskich przygotowano lokalne środowisko wieloregionowe (`start-local.ps1` z PR 92).

F4 — Eksperyment porównawczy. W ramach PR 92 zrealizowano dedykowany moduł eksperymentu obciążeniowego w katalogu `experiments/f4/`. Przygotowano syntetycznego klienta asynchronicznego oraz skrypty runnera (`run.py`, `run-f4.ps1`), które porównują wariant A (pojedynczy region EU) z wariantem B (dwa regiony EU/US). Scenariusze obejmują tworzenie pokojów, dołączanie, czat WebSocket oraz propagację listy. Narzędzia mierzą opóźnienia (p50, p95), kody odpowiedzi, błędy i zachowanie pod obciążeniem, generując ustrukturyzowane raporty JSON i Markdown.

F5 — Próby awarii. Kontrolowane wyłączenie jednego regionu (zatrzymanie usługi backendu, odcięcie ruchu tunelu, zatrzymanie bazy), obserwacja drugiego i pomiar odzyskania: czas wykrycia, czas przełączenia ruchu, zachowanie klientów (frontend posiada heartbeat WS 30 s i automatyczne ponowne łączenie) oraz stan pokoi po powrocie regionu (udokumentowane w `docs/operations/runbook-regions.md`).

F6 — Analiza wyników. Raport z wnioskami: kiedy dwa regiony pomagają, ile kosztują (obliczenia, baza, transfer międzyregionalny, sonda regionu) i jakie mają ograniczenia (w tym podział społeczności graczy między regiony), zasilany danymi z eksperymentu F4 i prób F5.

F7 — Obserwowalność i inspekcja MCP. W ramach PR 96, 97 i 99 rozszerzono zakres platformy o zaawansowane narzędzia diagnostyczne i telemetryczne:

- **Centralna obserwowalność (PR 99):** Uruchomiono maszynę `playlink-ops` ze stosem VictoriaLogs (logi), VictoriaMetrics (metryki) i Grafana (`deploy/observability.sh`). Kontenery Vector w każdym regionie przesyłają logi JSON z identyfikatorem `request_id` i zdarzeniami biznesowymi, a cAdvisor zbiera metryki zużycia zasobów. Błędy frontendu trafiają do logów przez `/api/client-errors`. Panel Grafany chroniony jest przez Cloudflare Access.
- **Serwer MCP (PR 96, 97, 99):** Wdrożono bezpieczny serwer MCP w trybie *read-only* (`backend/mcp_server/`) z transportem stdio i SSE. Oferuje narzędzia inspekcji gier, pokojów, zdrowia (`check_platform_health`) oraz zasoby per region (`playlink://{region}/*`) z agregacją multi-region. Dostępne są też narzędzia kwerend logów LogsQL (`search_logs`) oraz metryk PromQL (`query_metrics`).

6.2 Poza zakresem

- Replikacja bazy i pokoi między regionami (aktywna–aktywna) oraz współdzielone lobby ponad regionami.
- Uruchomienie trzeciego regionu w tych 10 tygodniach. Szablon i lista adresów mają go przyjąć bez zmiany architektury; samo wdrożenie zostaje poza zakresem, o ile po F1–F6 nie zostanie czas.
- Przeniesienie aplikacji na Cloudflare Containers albo bazy na D1 (uzasadnienie w sekcji 11.3).
- Osobna usługa balansowania (Azure Traffic Manager, Cloudflare Load Balancing) na ścieżce ruchu. Zostają opisane jako odpowiednik produkcyjny sondy w przeglądarce.
- Autoskalowanie produkcyjne, CDN poza podstawowym, WAF i pełna zgodność z regulacjami (np. RODO dla danych w USA) — odnotowane jako ryzyko.
- Matchmaking oparty na umiejętnościach, płatności, aplikacje mobilne.
- Zmiana mechanizmu uwierzytelniania (BIP39/ECDSA pozostaje bez zmian).

7 Kluczowe wymagania biznesowe

ID	Wymaganie	Filar
WB-1	Każdy region obsługuje pełny przepływ: logowanie, tworzenie, dołączanie, czat, lista na żywo	F1 (PR 91)
WB-2	Przy pierwszym wejściu użytkownik trafia do szybszego żywego regionu, zanim otworzy się lobby	F2 (PR 91)
WB-3	Zapisany region zostaje. Szybszy drugi region jest propozycją (Stay/Switch). Awaria zapisanego regionu przełącza na drugi. Ręczna zmiana działa	F2 (PR 91, 92)
WB-4	Oba środowiska uruchamia jeden szablon lub skrypt, z automatycznym wdrożeniem wydań pull-based	F3 (PR 91)
WB-5	Porównanie 1 vs 2 regionów odbywa się według opisanego, powtarzalnego scenariusza w `experiments/f4/`	F4 (PR 92)
WB-6	Awaria jednego regionu nie wyłącza drugiego, a procedury odzyskania są udokumentowane w runbooku	F5
WB-7	Wnioski zawierają koszt i ograniczenia, nie tylko korzyści	F6
WB-8	Konto użytkownika działa w obu regionach bez ponownej rejestracji	F1/F2
WB-9	Centralna obserwowalność (logi, metryki, Grafana) oraz inspekcja platformy przez asystentów AI (MCP)	F7 (PR 96, 97, 99)

8 Główne procesy biznesowe

P1. Wejście użytkownika. Adres strony otwiera jeden z regionów i stamtąd przychodzi strona ze skryptem sondy. Skrypt mierzy ping i dopiero potem otwiera lobby w szybszym żywym regionie. Kolejna wizyta wraca do zapisanego regionu. Logowanie to podpis tym samym kluczem. Pierwszy podpis w regionie zakłada tam konto z losową nazwą.

P2. Zmiana regionu. Gracz zmienia region ręcznie albo przyjmuje propozycję Stay/Switch, gdy drugi region jest szybszy. Adresy API i WS przełączają się, a lista pokojów jest wczytywana z nowego regionu. Sam pomiar regionu nie zmienia bez zgody użytkownika.

P3. Gra w lobby. Tworzenie i dołączanie do pokoju, czat i RSVP działają w obrębie wybranego regionu; pokoje wygasają jak dotychczas.

P4. Wdrożenie i aktualizacja. Publikacja wydań na GHCR uruchamia pull-based timer systemd na maszynach Azure (`deploy/update.sh`), który pobiera obrazy i bezpiecznie aktualizuje regiony (najpierw EU, po weryfikacji zdrowia US).

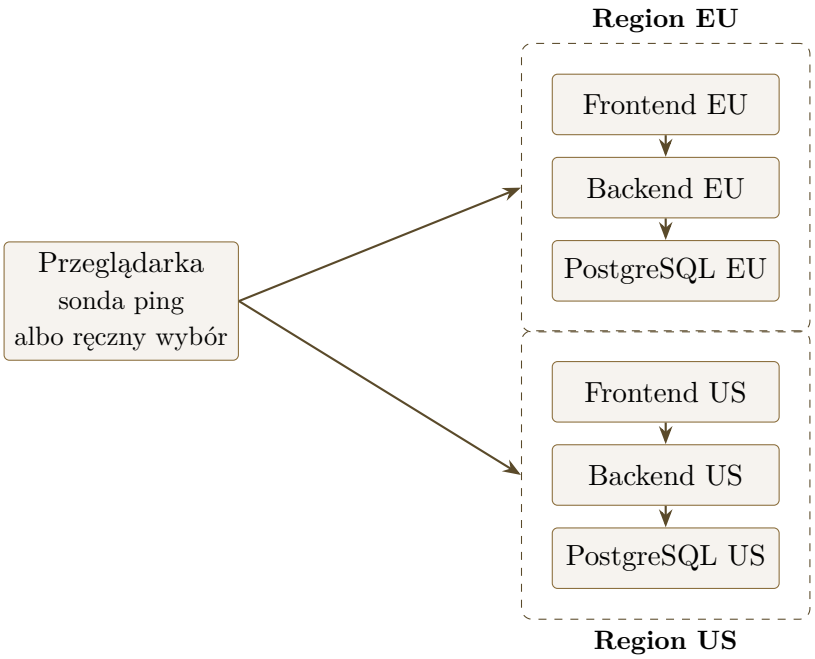
P5. Eksperyment. Uruchomienie scenariuszy obciążeniowych z katalogu `experiments/f4/` dla wariantu 1 i 2 regionów; rejestracja czasów p50/p95, błędów i synchronizacji WebSocket.

P6. Awaria i odzyskanie. Wyłączenie regionu, automatyczne przekierowanie graczy na zdrowy region i weryfikacja procedur awaryjnych z runbooka.

P7. Analiza. Zestawienie metryk, kosztów maszyn w Azure oraz wniosków architektonicznych.

P8. Obserwowalność i inspekcja MCP. Zbieranie logów i metryk przez Vector/cAdvisor

do VictoriaLogs/VictoriaMetrics oraz inspekcja stanu i telemetrii przez serwer MCP.



Rysunek 1: Docelowa topologia. Pierwsza strona przychodzi z jednego z dwóch regionów. Skrypt na niej wybiera lobby i dalszy ruch idzie tylko tam. Trzeciego serwera nie ma.

9 Wymagania niefunkcjonalne

Wymagania z raportu końcowego (WN-01 do WN-12) pozostają w mocy dla *każdego* regionu. Poniżej wymagania nowe lub zmienione. Wartości docelowe są **hipotezami** do potwierdzenia w eksperymencie F4/F5.

ID	Wymaganie	Miara (cel do weryfikacji)
WN-13	Opóźnienie REST dla użytkownika w regionie własnym	p95 $<$ 200 ms (jak WN-01)
WN-14	Aktualizacja listy pokojów po zdarzeniu	$<$ 1000 ms (jak WN-02), w obu regionach
WN-15	Wykrycie awarii regionu i przekierowanie nowych użytkowników	Czas wykrycia i przełączenia zmierzony; cel ustalany po pierwszej próbie
WN-16	Odtwarzalność środowiska	Wdrożenie z pustego konta/regionu jednym poleceniem (zrealizowane: `deploy/deploy.sh`, `deploy/azure-vm.sh`)
WN-17	Spójność konfiguracji między regionami	Te same wersje obrazów z GHCR i parametryzacja w plikach `.run/<region>.env`
WN-18	Kontrola kosztów	Budżet i alarm kosztowy; wynik kosztu w raporcie F6
WN-19	Obserwowalność	Centralny stos VictoriaLogs/Metrics i Grafana (PR 99), logi JSON z `request_id`, metryki kontenerów i serwer MCP
WN-20	Bezpieczeństwo	Sekrety poza repozytorium, baza bez publicznego dostępu, ruch wyłącznie przez Cloudflare Tunnel, TLS/WSS

10 Integracje

Poniżej są tylko usługi, z których projekt korzysta. Odpowiedniki, których nie uruchamiamy, są w sekcji 11.3.

Element	Rola w rozwiązaniu
Maszyny	Trzy maszyny Azure Virtual Machines, rozmiar `Standard_B2ats_v2`, Ubuntu: dwie dla regionów aplikacji oraz jedna dla centralnego stosu obserwowalności `playlink-ops`.
Stos na maszynie	`compose.yaml`: frontend, backend, PostgreSQL i agenci (Vector, cAdvisor) w kontenerach; `compose.observability.yaml` dla maszyny ops.
Przepis regionu	Parametryzowane skrypty `deploy/deploy.sh` oraz konfiguracja per region w `.run/<region>.env` (PR 91).
Sekrety	`JWT_SECRET`, hasła bazy i `OBS_PASSWORD` w `deploy/site.env` na maszynach; brak sekretów w repozytorium.
Cloudflare DNS i tunel	Nazwy regionalne (`playlink-<reg>`), adres główny i tunel `playlink-ops`. Ruch wychodzący przez `cloudflared`; brak otwartych portów publicznych na maszynach VM.
Ochrona i certyfikat	Ruch wchodzi na Cloudflare (DDoS, TLS). Panel Grafany (`playlink-grafana.<domain>`) zabezpieczony przez Cloudflare Access (PR 99).
Cache plików statycznych	Cloudflare buforuje pliki JS, CSS i obrazy; sonda, API, WebSocket oraz telemetria omijają cache.
Wybór regionu	Sonda w przeglądarce z mechanizmem propozycji Stay/Switch (`RegionProposalBanner`, PR 92) oraz ręczny przełącznik.
Wdrożenie i aktualizacja	Publikacja wydań na GHCR (`release.yml`); maszyny Azure pobierają i wdrażają nowe wersje w trybie pull-based (`deploy/update.sh`) co 2 minuty (EU najpierw, potem US) z automatycznym rollbackiem (PR 91).
Obserwowalność	Grafana, VictoriaLogs (LogsQL) i VictoriaMetrics (PromQL), agenci Vector i cAdvisor (PR 99).
Serwer MCP	Bezpieczny serwer inspekcji i telemetryczny dla asystentów AI (`backend/mcp_server`) z obsługą narzędzi logów i metryk (PR 96, 97, 99).
Testy obciążeniowe	Dedykowany runner syntetycznego obciążenia w `experiments/f4/` (PR 92).

11 Założenia i ograniczenia

11.1 Założenia

- Regiony: Europa (Frankfurt, spójny z domyślnym `eu-central`) i USA (wschodnie wybrzeże, spójny z `na-east`). *Do potwierdzenia.*
- Pokoje są krótkotrwałe, więc ich utrata przy awarii regionu jest akceptowalna (RPO dla lobby bliskie zeru); trwałe dane (konta) są odtwarzalne z podpisu, bo tożsamość jest bezhasłowa i przechowywana wyłącznie po stronie użytkownika (non-custodial).
- Koszty pokrywają kredyty dla studentów lub niski własny budżet; eksperyment używa małych instancji i jest wyłączany po testach (sekcja 11.3).
- Frontend i backend dają się wdrożyć niezależnie (WN-10 z raportu).

11.2 Ograniczenia

- Stan realtime w pamięci procesu (`ConnectionManager`, `RoomChatManager`, `WS_LIMITS`, limiter slowapi) wymusza jedną instancję backendu na region albo wprowadzenie wspólnego brokera — to poza zakresem.
- Limit REST (`10/minute`) jest liczony po adresie IP. Ruch wchodzi przez Cloudflare, więc backend czyta adres gracza z nagłówka `CF-Connecting-IP`. Bez tego wszyscy wyglądaliby jak jeden adres Cloudflare.
- JWT w ciasteczku `httpOnly` i `ORIGIN` muszą pasować do domeny regionalnej; wymaga to dopracowania konfiguracji frontendu.
- Nazwa użytkownika żyje w bazie regionu. Pierwsze logowanie w regionie dostaje losową nazwę `user_` i osiem znaków, tak jak dziś. Gracz może ją zmienić, jeśli w tym regionie jest wolna. Ta sama nazwa z drugiego regionu może być zajęta i wtedy zostaje przy losowej.
- Logowanie w drugim regionie to drugi podpis tym samym kluczem z przeglądarki. Token z pierwszego regionu nie przenosi sesji między osobnymi adresami. Wspólny `JWT_SECRET` zostaje, żeby token dało się zweryfikować tak samo.
- Czas i budżet zespołu są ograniczone terminami modułu.

11.3 Wybór chmury i koszty

Stos aplikacji (kontenery i PostgreSQL) stawiamy na **Azure**, bo program Azure for Students daje kredyty bez karty kredytowej i na dłuższy czas. **Alternatywa dla stosu: AWS**, jeśli weryfikacja studencka Azure się nie powiedzie lub kredyty okażą się niewystarczające. Konto studenckie jest już aktywne: 88 USD do 14.10.2027. **Cloudflare** trzyma domenę, przyjmuje ruch graczy i tunelem oddaje go maszynie. Lobby działa na maszynie, nie w Cloudflare.

11.3.1 Dlaczego nie ma trzeciego serwera

Dwa regiony nie wymagają maszyny, która stoi pośrodku i rozdziela ruch. Taka maszyna byłaby wąskim gardłem: każdy gracz, także ten blisko swojego regionu, musiałby przejść przez jeden punkt, a awaria tego punktu wyłączałaby oba regiony.

Wpisanie adresu nie łączy od razu z właściwym lobby. Nazwa jest na Cloudflare, a tunel tej nazwy kończy się na jednej z dwóch maszyn. Stamtąd przeglądarka dostaje stronę oraz skrypt. Oba regiony mają ten sam skrypt. Skrypt mierzy czas odpowiedzi obu nazw i dopiero potem zmienia adres w pasku na szybszy region. Użytkownik widzi lobby już po tym pomiarze. Ręczny przełącznik zostaje. Gdy wybrany region przestaje odpowiadać, przeglądarka przechodzi na kolejny zdrowy adres. Dodanie regionu to nowy adres na liście i ten sam stos z innym parametrem. Trzeci serwer, który tylko oddawałby ten skrypt, nic nie wnosi.

Pomiar przechodzi przez Cloudflare, więc liczba milisekund jest wyższa niż przy gołym połączeniu z maszyną. Porównanie dwóch regionów zostaje uczciwe, o ile sonda, API i WebSocket nie są serwowane z cache. W raporcie F4 zapisujemy, że mierzona ścieżka to przeglądarka, węzeł Cloudflare, tunel, maszyna.

11.3.2 Co bierzemy z Cloudflare

Wszystko poniżej jest w planie darmowym. Płatnego load balancera nie uruchamiamy.

- **Tunel i DDoS.** `cloudflared` na każdej maszynie. Gracz nie łączy się z publicznym adresem maszyny. Atak na nazwę strony zatrzymuje się na sieci Cloudflare.
- **Certyfikat TLS** dla nazw regionu. Przeglądarka rozmawia po HTTPS i WSS.
- **Cache plików statycznych.** JS, CSS i obrazy biorą się z węzła blisko gracza i nie obciążają małej maszyny. Sonda, API i WebSocket mają regułę obejścia cache.
- **Kompresja i HTTP/3.** Są włączone, bo przyspieszają pobranie strony. Wznowienie 0-RTT zostaje wyłączone, żeby powtórzona sonda nie była sztucznie cichsza od pierwszej.
- **WebSocket.** Czat i lista pokoi przechodzą przez tunel. Istniejący heartbeat co 30 s zostaje, bo bezczynne połączenie Cloudflare potrafi zamknąć.

Poza zakresem zostają funkcje, które albo kosztują, albo psują pomiar i testy: Argo Smart Routing, Load Balancing, tryb walki z botami, Rocket Loader, kontenery Cloudflare i baza D1.

11.3.3 Czego nie przenosimy na Cloudflare

- **Kontenery.** Cloudflare Containers nie mają planu darmowego. Wchodzą w Workers Paid (5 USD miesięcznie) i są rozliczane za czas działania. Wliczone 25 GiB-godzin nie wystarcza na dwa zawsze włączone lobby: nawet mały kontener działający cały miesiąc przekracza ten próg, a WebSocket ma pozostać otwarty. Kontenery Cloudflare są liczone od obsłużenia żądania do uśpienia, co nie pasuje do lobby trzymanego w pamięci procesu.
- **Baza D1.** D1 jest w planie darmowym (5 GB, limity odczytów i zapisów na dobę), ale jest to SQLite, nie PostgreSQL. PlayLink już używa PostgreSQL w obecnym stosie. Przepisanie warstwy danych byłoby nową aplikacją, a nie rozbudową tej, którą mamy. Hyperdrive tylko przyspiesza połączenie z zewnętrznym Postgresem; nie zastępuje bazy w regionie.

11.3.4 Kontenery, sekrety, przepis

Na każdy region przypada stos kontenerów: frontend, backend, PostgreSQL oraz agenci telemetryczni (Vector, cAdvisor). Baza jest kontenerem Postgres w compose (`compose.yaml`). Usługi Azure Database for PostgreSQL nie uruchamiamy. Obrazy budowane są wieloarchitekturowo i publikowane w GHCR. Sekrety (`JWT_SECRET`, hasła bazy, `OBS_PASSWORD`) nie wchodzą do repozytorium i nie idą do Azure Key Vault. Leżą w plikach środowiskowych na maszynie (`deploy/site.env`), a wdrożenie nie wymaga otwartych portów SSH dla CI.

Przepis regionu to pliki w repozytorium: `compose.yaml` plus plik parametrów `.run/<region>.env`. Jedno polecenie `deploy/deploy.sh eu` stawia stos regionalny, to samo polecenie z parametrem `us` stawia drugi region. Aktualizacje realizowane są w trybie pull-based przez timer systemd (`deploy/update.sh`) odpytujący rejestr GHCR, co zapewnia pełną automatyzację bez ryzyka wycieku kluczy dostępowych. Płaci się wyłącznie za czas maszyny i dysk, nie za narzędzie. Maszyna centralnego monitoringu (`playlink-ops`) korzysta z tego samego wzorca (`compose.observability.yaml`).

Maszyna. Usługa nazywa się Azure Virtual Machines. Rozmiar to `Standard_B2ats_v2` (2 vCPU, 1 GiB RAM, AMD), obraz Ubuntu Server 24.04 LTS. Regiony: Germany West Central i East US. Dysk systemu: 64 GB Standard SSD, bo większy dysk Premium liczy się z kredytu osobno. Oferta obejmuje 750 godzin miesięcznie tego rozmiaru. Jedna maszyna włączona cały miesiąc mieści się w tym limicie, druga schodzi z kredytu 88 USD, więc po próbach obie się wyłącza. Gdy 1 GiB nie pomieści trzech kontenerów, rozmiar zmienia się na `Standard_B1ms` (2 GiB) albo `Standard_B2s` (4 GiB). Te większe rozmiary nie są w darmowych godzinach.

Dalsze dokumenty tego modułu mają opisywać wykonane kroki: co uruchomiono, jaki był wynik, jaki wniosek. Sam opis dostawcy nie zamyka fazy.

Usługa	**Dlaczego jej nie ma w tym etapie**
Azure Database for PostgreSQL	Baza już jest kontenerem Postgres w compose. Osobna usługa zjadłaby kredyt i zdublowała dane.
Azure Key Vault	Sekrety już obsługuje GitHub Actions i plik na maszynie.
Azure Traffic Manager	Wybór regionu robi skrypt i zmienia adres w pasku. Traffic Manager tylko podmienia odpowiedź DNS, nie chroni maszyny i potrafi przerzucić gracza na drugi region pod tą samą nazwą.
Cloudflare Load Balancing i Argo	Płatne. Tunel i skrypt robią ochronę oraz wybór regionu bez abonamentu.

Tabela 2: Usługi rozważane i świadomie pominięte.

	Azure for Students	**AWS**
Kredyty	100 USD na 12 miesięcy, odnawiane co roku dla studenta	Do 200 USD: 100 USD na start oraz po 20 USD za pięć zadań (uruchomienie instancji EC2, baza RDS, funkcja Lambda, zapytanie w Bedrock, ustawienie budżetu); plan darmowy wygasa po 6 miesiącach lub po wyczerpaniu kredytów
Karta kredytowa	Nie jest wymagana	Wymagana przy zakładaniu konta
Weryfikacja studenta	Poczta uczelniana; domena musi być rozpoznana	Nie dotyczy (kredyty dla nowych kont); AWS Educate osobno
Ryzyko	Uczelniana domena może nie zostać rozpoznana	Po wyczerpaniu kredytów płatne zasoby; wymagany budżet z alarmem

Tabela 3: Kredyty dla studentów (dane orientacyjne z publicznych opisów programów; przed założeniem konta zweryfikować na stronach dostawców).

Szacunek. Z kredytu schodzi czas drugiej maszyny i dyski. Jedna maszyna tego rozmiaru mieści się w 750 darmowych godzinach miesięcznie. Baza w kontenerze osobnego rachunku nie generuje. Cloudflare w zakresie powyżej jest w planie darmowym. Po próbach maszyny się wyłącza. Alarm budżetu ustawiamy, zanim druga maszyna zostanie włączona na dłużej. F6 wpisze kwotę z rzeczywistego rachunku.

Decyzja fazy 0 jest zamknięta: Azure for Students, maszyny `Standard_B2ats_v2`, baza w kontenerze, Cloudflare przy nazwach i tunelu. AWS zostaje tylko awaryjnie, gdyby kredyt się skończył przed końcem modułu.

12 Ryzyka

ID	Ryzyko	Wpływ	Reakcja
R1	Podział społeczności: gracze z różnych regionów nie widzą wzajemnych lobby	Wysoki	Ręczny wybór regionu; podgląd listy drugiego regionu jako rozszerzenie; ujęcie w F6
R2	Koszty przekraczają budżet lub kredyty (transfer, baza, kontrola stanu)	Średni	Małe instancje, budżet z alarmem, wyłączanie środowiska po testach; zapasowo drugi dostawca (AWS)
R3	Stan realtime w pamięci uniemożliwia skalowanie w obrębie regionu	Średni	Jedna instancja na region w zakresie; udokumentowane jako ograniczenie
R4	Różnice konfiguracji między regionami zafałszowują eksperyment	Wysoki	Jeden szablon, te same obrazy, porównanie konfiguracji przed testem
R5	Zła metodologia pomiarów (generator blisko serwera, krótki test, szum)	Wysoki	Generatory w obu kontynentach, powtórzenia, opis warunków
R6	Zbyt wolne wykrycie awarii przy domyślnych ustawieniach kontroli stanu	Średni	Zmierzyć i dostroić; wynik zapisać w F5
R7	Dane użytkowników w USA a wymagania prawne (RODO)	Średni	Przechowywane są tylko adresy tożsamości i nazwy; ocena w F6, nie wdrożenie zgodności
R8	Zakres ponad możliwości zespołu w 10 tygodniach	Średni	Priorytety: F1–F3 w pierwszej kolejności, F6 zawsze dostarczone

13 Kryteria sukcesu

Filar	Kryterium sukcesu	Dowód i stan realizacji
F1	Oba regiony przechodzą ten sam zestaw testów przepływu (pokój, dołączenie, czat, lista)	Spełnione: stosy EU/US za Cloudflare Tunnel, testy backendu i frontendu przechodzą (PR 91)
F2	Pierwsze wejście trafia do szybszego regionu; szybszy drugi region jest propozycją (Stay/Switch)	Spełnione: baner `RegionProposalBanner`, cookie `playlink.region`, failover przy awarii (PR 91, 92)
F3	Czyste wdrożenie obu środowisk jednym poleceniem; powtarzalna aktualizacja wydań	Spełnione: `deploy/deploy.sh`, obrazy GHCR i timer pull-based `deploy/update.sh` (PR 91)
F4	Zebrane pomiary wariantów A i B dla obu grup użytkowników, z opisem warunków	Spełnione: moduł `experiments/f4/`, raporty wyników p50/p95, błędy i WebSocket (PR 92)
F5	Co najmniej dwa scenariusze awarii wykonane, z czasem wykrycia i odzyskania	Scenariusze stopu backendu/tunelu i procedury runbookowe (`docs/operations/runbook-regions.md`)
F6	Raport z rekomendacją: kiedy opłaca się drugi region, jaki koszt i jakie ograniczenia	Podsumowanie wyników eksperymentu F4, kosztów maszyn Azure oraz wniosków
F7	Centralna obserwowalność oraz bezpieczna inspekcja stanu przez asystentów AI (MCP)	Spełnione: Grafana, VictoriaLogs/Metrics (PR 99), serwer MCP z narzędziami logów i metryk (PR 96, 97)

Hipotezy do sprawdzenia: H1 — dla użytkowników z USA wariant A (tylko Europa) nie spełnia WN-13, a wariant B tak. H2 — dla użytkowników z Europy wariant B nie pogarsza wyników względem A. H3 — po awarii jednego regionu drugi pozostaje w pełni użyteczny. Wynik negatywny też jest wartościowym wnioskiem F6.

14 Roadmapa produktu

Plan obejmuje 10 tygodni liczonych od startu prac. Architektura na ten etap jest ustalona. Konto studenckie jest aktywne (88 USD, ważność do 14.10.2027). Stos staje na maszynach wirtualnych `Standard_B2ats_v2`. Reszta tygodni jest wykonaniem, pomiarem i zapisem wyniku.

14.1 Dziennik etapu

Każdy tydzień zamyka krótki wpis, ten sam dla wszystkich faz. Wpis ma cztery pola: co było zaplanowane, co zostało uruchomione, jaki był wynik (liczba, czas, zrzut albo komunikat błędu) i jaki z tego wniosek, także gdy coś nie działa. Sam opis dostawcy nie zamyka tygodnia. Z tych wpisów składa się materiał dla prowadzącego oraz raport F6 (prowadzony na bieżąco w `docs/dziennik-etapu.md`).

Tydzień	Filar	Praca zrealizowana	Wpis w dzienniku i rezultaty
1	decyzje	Konto Azure for Students aktywne; maszyny `Standard_B2ats_v2` z Ubuntu; konfiguracja domen i tuneli Cloudflare	Zrzut salda studenckiego, specyfikacja maszyn, domeny w Cloudflare
2–4	F1, F3	Wdrożenie wieloregionowe EU/US za tunelami Cloudflare; potok budowania wydań GHCR i aktualizacja pull-based (`deploy/update.sh`)	Działające dwa regiony, healthchecki, runbook operacyjny (`docs/operations/runbook-regions.md`) (PR 91)
5–7	F2, F4	Lokalne środowisko wieloregionowe (`start-local.ps1`), propozycja Stay/Switch (F2) w UI oraz realizacja eksperymentu obciążeniowego F4 w `experiments/f4/`	Pomiary p50/p95 wariantu 1 vs 2 regionów, weryfikacja braku samoczynnego przeskakiwania lobby (PR 92)
8	MCP	Implementacja serwera Model Context Protocol: bezpieczna inspekcja gier i pokoi, dynamiczne regiony RFC 6570 (`backend/mcp_server`)	Narzędzia i zasoby MCP przetestowane lokalnie i w CI (PR 96, PR 97)
9	Obserw.	Centralny stos obserwowalności na maszynie `playlink-ops` (VictoriaLogs, VictoriaMetrics, Grafana) oraz narzędzia logów i metryk w MCP	Dashboard *PlayLink regions*, logi z `request_id`, narzędzia `search_logs` i `query_metrics` w MCP (PR 99)
10	F5, F6	Próby awarii, podsumowanie kosztów rzeczywistych, raport końcowy i odbiór projektu	Komplet wyników w dzienniku etapu, wnioski architektoniczne i obrona projektu

Poza horyzontem tego etapu: podgląd lobby drugiego regionu, wspólny broker zdarzeń i wdrożenie trzeciego regionu z gotowego szablonu.

15 Korzyści biznesowe

- **Dla graczy:** niższe opóźnienia i dostęp do lobby blisko siebie lub w drugim regionie na życzenie.
- **Dla ciągłości usługi:** awaria jednego regionu nie wyłącza całej platformy.
- **Dla zespołu:** powtarzalne wdrożenie zamiast ręcznej konfiguracji jednego hosta; łatwiejsze dodanie kolejnego środowiska.
- **Dla decyzji:** dane, czy drugi region jest opłacalny, zanim zostaną poniesione stałe koszty.
- **Dla oceny modułu:** jasno rozdzielone, mierzalne rezultaty oraz nabyta praktyka z chmury, IaC i testowania odporności.

16 Załączniki

Z1. Raport końcowy PlayLink (`prezentacje/raport_koncowy/raport.pdf`).
Z2. Podręcznik wdrożenia i operacji wieloregionowych: `docs/operations/runbook-regions.md`.
Z3. Dokumentacja obserwowalności (Grafana, VictoriaLogs, VictoriaMetrics): `docs/operations/observability.md`.
Z4. Dokumentacja serwera Model Context Protocol (MCP): `backend/mcp_server/README.md`.
Z5. Moduł i raport eksperymentu obciążeniowego 1 vs 2 regiony: `experiments/f4/`.
Z6. Dziennik postępów etapu: `docs/dziennik-etapu.md`.
Z7. Potok wydań i publikacji obrazów: `.github/workflows/release.yml`.
Z8. Lista lokalizacji lobby: `backend/config.py` (`LOBBY_LOCATIONS`).